NeuroGenUI: Multimodal Sketch-and-Voice Generation of Editable, Validated User Interfaces through a Structured UI Intent Representation

Author One, Author Two, Author Three
Department of Computer Science and Engineering
Institution Name, City, Country
{author1, author2, author3}@institution.edu

Abstract—We present NeuroGenUI, a multimodal framework that generates editable React user interfaces from a hand-drawn sketch and spoken instructions, and refines them with further voice commands. Instead of a single image-to-code call, the system separates perception, interpretation, fusion, generation and validation around an explicit JSON UI Intent Representation: OpenCV proposals are labelled by a vision model, speech is transcribed by Whisper and converted into typed edit operations, a deterministic geometry-aware compiler emits React code, and a headless-browser validator drives a targeted repair loop. We evaluate every stage on 40 labelled synthetic sketches and 80 synthetic spoken commands, against a direct sketch-to-UI baseline using the same model and measurement code. On rendered UIs, the pipeline detects the sketched components with F1 42.9% versus 25.6%, passes 94.8% of specified interactions versus 86.2%, and applies refinements in 2.1 s versus 15.4 s while preserving 99.8% of unaffected components; requirement satisfaction and refinement success are at parity with the baseline. Fault injection shows the validator detects 96.8% of injected faults and the repair loop restores 73.5%. An initial evaluation exposed defects that were fixed on the same test set, so results are development-set figures on synthetic data. Code, dataset and evaluation scripts are provided.

Index Terms—sketch-to-code, multimodal interaction, speech interfaces, UI generation, large language models, program validation, automated repair

I. Introduction

Designers usually start a user interface (UI) with a rough sketch on paper or a whiteboard. Turning that sketch into working front-end code is slow and repetitive, and the sketch itself only says where things are: it does not say what a field validates, where a button navigates, or which colour a primary action should have. People state that kind of intent naturally in speech ("when the login button is clicked, show an error if the fields are empty"), not in drawings.

Recent vision–language models can turn a screenshot or sketch into code in a single call [1]. That single-call approach has three practical weaknesses for interactive design. First, its intermediate understanding is hidden inside the model, so detection, interpretation and generation cannot be measured or corrected separately. Second, every modification regenerates the whole program, which is slow and can silently change parts the user did not ask to change. Third, nothing checks that the produced UI actually renders, contains what was sketched, or behaves as requested.

This paper presents NeuroGenUI, a multimodal framework that combines a hand-drawn sketch with spoken instructions through an explicit, structured UI Intent Representation (IR). Each stage is separate and measurable: OpenCV proposes candidate components and a vision model labels them (hybrid detection); Whisper [2] transcribes speech; a language model interprets the transcript into typed edit operations without ever writing code; a deterministic fusion step merges sketch geometry with spoken semantics into the IR; a deterministic compiler emits an editable React project; a headless-browser validator checks visual and functional properties; and a targeted repair loop fixes the specific component that failed. Spoken refinements are applied as operations on the existing IR, so unaffected components are preserved by construction.

The contributions of this work are:

- A staged sketch + voice → IR → React architecture in which no stage is a single image-to-code call and every stage is timed and evaluated in isolation.
- A JSON UI Intent Representation that carries type, content, geometry, style, hierarchy, relations and event/action semantics, together with a typed operation language for voice-guided refinement.
- A validation and targeted repair loop that runs the generated code in a real browser and maps failing checks back to IR components.
- An open evaluation protocol with 40 labelled sketches across 10 screen categories and 80 spoken commands across eight edit categories, and a like-for-like comparison against a direct sketch→AI→UI baseline using the same model, the same test cases and the same measurement code.

We do not train a new foundation model; the contribution is the system architecture and its measurement. All numbers in this paper are produced by the released evaluation scripts.

II. Related Work

Sketch-based UI design. SILK [3] showed early that electronic sketching supports the exploratory stage of UI design and that sketched widgets can be recognised and made interactive. Later systems learned the mapping from images to code: pix2code [4] generated a domain-specific language from screenshots with a CNN–LSTM, ReDraw [5] classified detected mobile components and assembled them into app code, and several Sketch2Code systems [6], [7] produced web pages from paper mockups with object detection and heuristics. Large UI corpora such as Rico [8] enabled data-driven design tools. These systems are vision-only: behaviour, navigation and styling intent cannot be expressed in the sketch and are therefore absent from the output.

LLM-based front-end generation. General multimodal models [9] can produce HTML/React from an image in one call, and Design2Code [1] benchmarks this ability on real web pages. Our baseline follows this direct approach. Our work differs in that the model is used only for perception (labelling proposals, reading handwriting) and interpretation (speech to typed operations), while code generation, refinement and repair are deterministic operations on an explicit representation.

Multimodal and voice interaction. Combining speech with a spatial channel goes back to "Put-That-There" [10]. Modern speech recognisers such as Whisper [2] make spoken commands practical. NeuroGenUI uses speech for the information a sketch cannot carry: semantics, behaviour and edits to an existing design.

Iterative refinement and repair. Self-refinement methods let language models critique and revise their own output [11]. Our repair loop is instead grounded in executable checks: the generated UI is rendered in a headless browser, failures are attributed to specific IR components, and only those components are changed before revalidation.

III. System Architecture

Fig. 1 shows the architecture and Fig. 2 the interaction flow. The stages are independent modules with fixed interfaces, each timed separately by the orchestrator (upload, analyze_sketch, transcribe, interpret, fuse, codegen, validate, repair, save).

A. Sketch Understanding

OpenCV proposals. The sketch is converted to grayscale, binarised with adaptive thresholding and closed morphologically so that wobbly, non-closed hand-drawn strokes join into contours. Contours are extracted with their hierarchy [12], [13], approximated by polygons and classified by geometry: small squares become checkbox candidates, small circles radio candidates, wide short rectangles input or button candidates, rectangles crossed by diagonals image placeholders, long thin strokes dividers, and clusters of small strokes handwritten text.

Vision labelling. The candidate boxes are drawn with numeric tags onto a copy of the sketch, and a vision model (GPT-4.1-mini) receives this annotated image. Under a JSON schema it assigns each numbered box a component type from the IR vocabulary, transcribes its handwritten text, and adds components the proposals missed. The model therefore classifies and reads but does not localise from scratch.

Merge. Vision labels are attached to proposals, added components are merged by IoU, and a containment tree assigns parents. Three modes are available for ablation: cv (geometry only), vision (the model alone, boxes included) and hybrid (the proposed combination).

B. Speech and Intent Understanding

Speech recorded in the browser is transcribed with Whisper (whisper-1). A language model (GPT-4.1-mini, temperature 0, JSON-schema output) converts the transcript into a list of typed operations: create, add, remove, modify, move, resize, style, interaction and navigation. When an IR exists, the model sees a compact outline of it (id, type, label, parent) so it can name targets directly. An operation carries a target reference, an optional new component, a relative position (below, above, left_of, right_of, inside), property and style changes, or an event with actions. The model never writes code.

C. UI Intent Representation

The IR is a JSON document with pages, each holding a component tree. Every node has a stable id, a type (page, container, card, form, header, navbar, nav item, heading, text, link, button, input, textarea, checkbox, radio, select, image, list, divider, icon), a label, type-specific properties (input type, placeholder, required flag, options), style (variant, colour, size, width, alignment), container layout, a normalised bounding box from the sketch, relations to other nodes, and events. Events bind a trigger (click, submit, change) to an ordered list of actions: validate_required, validate_format, show_message, navigate, toggle_visibility and set_text.

D. Multimodal Fusion

Fusion is deterministic. The sketch decides where things are: components are grouped into rows by vertical overlap, containment (by box centre) gives parents, side-by-side columns are detected, and drawn plain containers become cards or forms. The voice description decides what they are: input types, required flags, events and the page title. Matching runs in two passes. First, spoken and sketched components with near-identical labels are paired globally, and text-like types count as compatible. Second, each remaining spoken component is matched by type compatibility and reading order between its already-anchored neighbours. A spoken component whose label matches a drawn one is never inserted a second time. When drawn and spoken text disagree, the drawn text is

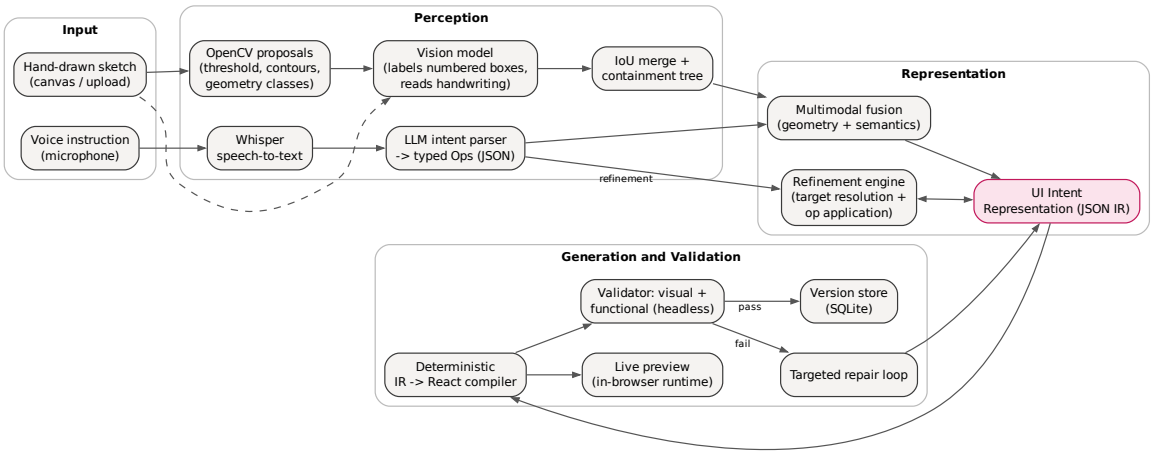


Fig. 1. NeuroGenUI architecture. Perception (OpenCV proposals, vision labelling, Whisper, LLM intent parsing) feeds a deterministic fusion step that produces the JSON UI Intent Representation (IR). The IR is compiled to React, rendered live, validated in a headless browser and repaired; voice refinements edit the IR rather than regenerating code.

kept and the voice contributes properties and behaviour. Unmatched spoken components are inserted next to their spoken neighbours, and unmatched sketched components are kept. Before fusion, the analysis is cleaned: fragments of one handwritten word are merged, duplicate detections of the same text are removed, and labels that contradict the drawn shape fall back to the geometric class.

E. Code Generation and Live Preview

A deterministic compiler translates the IR into a React project: one reusable component per IR type (Button, Field, Heading, Card, …), one component per page, a state-based router, and event handlers that call a small action runtime. The compiler is geometry-aware: the page frame (phone or desktop) follows the sketch's aspect ratio, and each node's normalised sketch box becomes widths and offsets proportional to its parent, vertical spacing proportional to the drawing, and drawn heights for images, buttons and boxes. All of this is expressed with flex rows and columns that wrap on narrow screens rather than absolute positioning, so the code stays responsive and readable. Explicit user styles always override geometry. The output is a runnable Vite project that the user can export and edit, plus a single-script form that the browser runtime transpiles with Babel and renders live in an iframe. Because the compiler is deterministic, the same IR always yields the same code, and a local IR edit yields a local code change.

F. Voice-Guided Refinement

A spoken refinement is transcribed and interpreted into operations against the current IR. Targets are resolved by the id the model supplied when it is valid, otherwise by a deterministic resolver that scores type keywords, fuzzy label similarity, ordinals ("the second button") and position words. Application is pure: the IR is deep-copied, only targeted nodes change, untouched nodes stay byte-identical, and every operation reports success or a reason for failure. The changed IR is recompiled and the preview refreshes. Every result is stored as a new version with its transcript, operations, IR, code, validation report and stage timings, so iterative refinement can be inspected and restored.

G. Validation and Targeted Repair

The validator loads the generated code into the same runtime page the user sees, inside a headless browser (Playwright, Chromium or Edge). Visual checks verify that every IR node is rendered and visible, that sibling order matches the IR and the sketch, that sketched components are covered, and the overlap between rendered boxes and sketch boxes. Functional checks capture runtime and console errors, click every button, type into every input, and simulate every event to assert its effect (an error message for empty required fields, the target page after navigation, a visibility change after a toggle).

The repair loop maps each failing check to the responsible IR node and applies a targeted fix: a missing sketched component is re-inserted from the analysis, out-of-order siblings are reordered, a hidden or zero-size node has its style reset, a node missing from the DOM or a runtime error triggers recompilation in a safe mode, and a failing interaction has its handler rebuilt. The UI is then revalidated, up to three iterations. Repairs never undo an explicit user edit: removed and moved components are recorded in the IR metadata and excluded from coverage and order repairs.

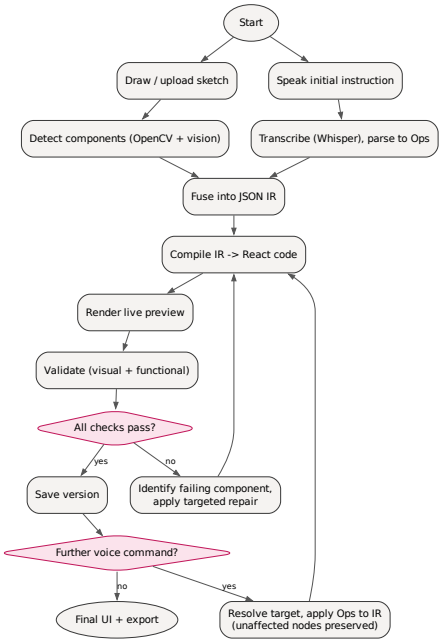


Fig. 2. Interaction flow with validation, targeted repair and iterative voice refinement.

IV. Experimental Setup

A. Dataset

Sketches. We built 40 UI sketches, four for each of 10 screen categories: login, registration, dashboard, profile, e-commerce, banking, booking, contact, settings and landing page. Each screen is specified as a component tree with varied layouts (navigation bars, multi-column rows, cards, image placeholders, radio and checkbox groups, selects, multiple buttons). It is rendered in a hand-drawn style with jittered, overshooting multi-segment strokes, slightly rotated and non-closed rectangles, varying pen width, three handwriting fonts and paper noise. Ground truth (type, label, normalised box and parent for 490 components) comes from the specification. Each sketch also has a natural spoken description used in the generation step, plus the interactions and requirements that description implies. The sketches are synthetic; Section VI-C discusses what this implies.

Voice commands. We wrote 80 commands, ten for each of add, remove, modify, move, resize, style, interaction and navigation. They include all seven example commands of the project specification (e.g., “Make the login button blue”, “Add a forgot password link below the login button”). Each command is tied to a sketch, labelled with its expected operation, target and parameters, and has a machine-checkable post-condition evaluated on the rendered UI. Each command was synthesised to speech with a text-to-speech model, rotating six voices, so that Whisper transcribes real audio.

B. Baseline

The baseline is the direct sketch→AI→UI approach: one call to the same vision model (GPT-4.1-mini) receives the sketch and the spoken instruction and returns a complete React program. Modification sends the previous program and the command, and receives a regenerated program. The baseline prompt is a competent, fair prompt, and its output follows the same runtime contract, so the same renderer measures both systems.

C. Metrics

All metrics are computed by the same code for both systems. Measurement never uses the pipeline’s internal data-ngid attributes; components are recovered from the rendered DOM by tags, ARIA roles, input types and visual cues.

- Component detection: Hungarian matching [14] of predicted to ground-truth boxes (IoU $\geq$ 0.3). A match counts as a true positive when the types agree. Precision, recall and F1 are micro-averaged. It is measured on the analysis output (cv/vision/hybrid) and on the rendered DOM of each system.
- Voice command accuracy: Whisper word error rate (WER), plus the share of commands whose operation type, target and parameters are all interpreted correctly, from both the reference text and the Whisper transcript.
- Visual similarity: mean layout IoU between ground-truth components and their assigned rendered elements, and SSIM [15] between dilated Canny [16] edge maps of the sketch and the screenshot.
- Functional correctness: the share of specified interactions that work when simulated in the browser. Requirement satisfaction: the share of instruction-derived requirements (named components, input types, behaviours) that are met.
- Refinement success: the share of commands whose post-condition holds on the refined UI. Preservation: the share of non-target components unchanged in type, text, colour, size and font.
- Validation pass rate before and after repair, and a fault-injection study: deleted, hidden, zero-size and reordered nodes, broken events and injected runtime errors, scored by whether the validator detects each fault and whether repair removes it.
- Generation time per stage and end to end, and code quality: parse success, lines of code, components, reuse ratio (component usages per definition), longest function and ESLint messages.

D. Implementation

The backend is Python (FastAPI, OpenCV, Playwright, SQLite for projects and versions), and the studio front end is React. The models are GPT-4.1-mini for vision and intent and whisper-1 for speech, through the OpenAI API. The evaluation reran every stage in-process with cached

TABLE I
Headline results on the same 40 sketches and 80 commands (proposed vs. one-call baseline)

Metric	Proposed	Baseline	Unit
Component detection F1 (analysis vs. baseline DOM)	53.1	25.6	%
Component detection F1 (both on rendered DOM)	42.9	25.6	%
Whisper word error rate	4.0	–	%
Voice command accuracy (from Whisper)	91.2	–	%
Visual similarity: layout IoU ×100	39.9	28.3	IoU×100
Visual similarity: edge SSIM ×100	52.2	50.9	SSIM×100
Functional correctness	94.8	86.2	%
Requirement satisfaction	87.5	86.0	%
Refinement success rate	95.0	90.0	%
Preservation of unaffected components	99.8	98.2	%
Validation pass rate before repair	100.0	–	%
Validation pass rate after repair	100.0	–	%
Injected faults detected	96.8	–	%
Injected faults repaired	73.5	–	%
Generation time, mean (s)	15.5	24.0	s
Modification time, mean (s)	2.1	15.4	s
Generated code parses	100.0	100.0	%
ESLint messages per UI	0.05	0.03	count/UI
Component reuse ratio	1.9	1.0	usages/definition
Longest function (lines)	105.6	206.1	lines

raw model outputs, so metrics can be recomputed without further API calls. The full run made 480 model calls.

V. Results

All results come from one full run over the 40 sketches and 80 commands (480 cached model calls). Both systems use the same model, test cases and measurement code. Table I summarises every metric; the subsections discuss them. Differences on this test set are single-run, and several are small enough (a few commands out of 80) that we describe them as parity rather than wins.

A. Component Detection

Table II separates the sketch-analysis stage from what is actually rendered. At the analysis stage, the hybrid detector reaches F1 53.1% against 42.9% for OpenCV alone and 33.5% for the vision model alone, so neither component is sufficient. OpenCV localises well (mean IoU of matched boxes 0.72) but cannot read text or reliably tell types apart (type accuracy 52.9%); the vision model labels well (87.7% label accuracy) but localises loosely (mean IoU 0.52). Hybrid keeps OpenCV's boxes and the vision model's labels (95.3% label accuracy).

The like-for-like comparison with the baseline is on the rendered DOM, because that is the only place the baseline's components exist. There the proposed pipeline scores F1 42.9% against 25.6% for the baseline. A ground-truth oracle IR compiled by the same compiler scores 77.5%, which bounds what the compiler achieves with perfect input and suggests that most of the remaining gap comes from upstream detection errors rather than code generation.

B. Voice Command Accuracy

Whisper transcribes the synthesised commands with a corpus word error rate of 4.0%. From the Whisper transcript, 91.2% of commands are interpreted fully correctly (operation, target and parameters); from the reference text the figure is 93.8% (Table III). Per category, add, remove, style, interaction and navigation are interpreted almost

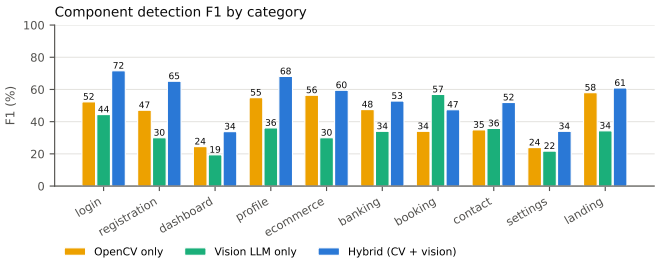


Fig. 3. Detection F1 per screen category for each analysis mode and for the rendered DOM of both systems.

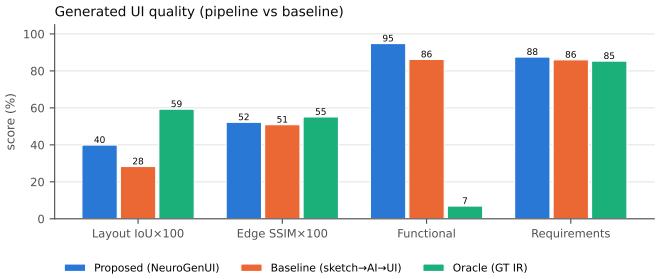


Fig. 4. Generated UI quality: pipeline, baseline and the ground-truth oracle.

perfectly; resize is the weakest (60% from the transcript) because quantities such as "a bit bigger" are ambiguous. Speech recognition errors cost about 2.6 percentage points relative to clean text (91.2% vs. 93.8%); we did not analyse which tokens were misrecognised.

C. Generated UI Quality

Table IV reports visual and functional quality. The generated UI renders in 100% of cases for both systems. Layout IoU against the sketch is 0.399 for the proposed pipeline and 0.283 for the baseline; the compiler's oracle ceiling is 0.593. Edge SSIM is nearly identical (0.52 vs. 0.51), so the two systems look equally similar to the sketch at the pixel-structure level; the gain is in component placement. Functional correctness is 94.8% vs. 86.2%, and the difference is almost entirely in navigation (92% vs. 72% of 25 specified interactions). Validation and message behaviours are equal. Requirement satisfaction is at parity (87.5% vs. 86.0%); the baseline is better at honouring specific input types (97.1% vs. 91.2%).

D. Voice-Guided Refinement

Table V shows refinement success and preservation per command category. The proposed pipeline applies 95.0% of commands successfully against 90.0% for the baseline (93.8% when the command goes through Whisper), a difference of four commands out of 80 that we do not claim as significant. The categories differ: the pipeline is better on add, style, navigation and resize; the baseline is better on modify and move (100% on both). The clearer difference is preservation. Because refinements

TABLE II
Component detection (micro-averaged; Hungarian matching, IoU ≥ 0.3, correct type)

Detector / output	Precision %	Recall %	F1 %	Type acc. of matched %	Mean IoU of matched
analysis: OpenCV only	40.7	45.3	42.9	52.9	0.724
analysis: Vision LLM only	32.3	34.9	33.5	69.0	0.521
analysis: Hybrid (CV + vision)	48.9	58.2	53.1	68.1	0.717
rendered DOM: Proposed (NeuroGenUI)	40.6	45.5	42.9	82.3	0.580
rendered DOM: Baseline (sketch→AI→UI)	25.9	25.3	25.6	74.1	0.580
rendered DOM: Oracle (GT IR)	79.0	76.1	77.5	95.5	0.692

TABLE III
Voice command interpretation accuracy (%)

Input	Commands	Op type	Target	Parameters	All correct
Ground-truth text	80	97.5	97.5	98.6	93.8
Whisper transcript	80	96.2	97.5	95.7	91.2

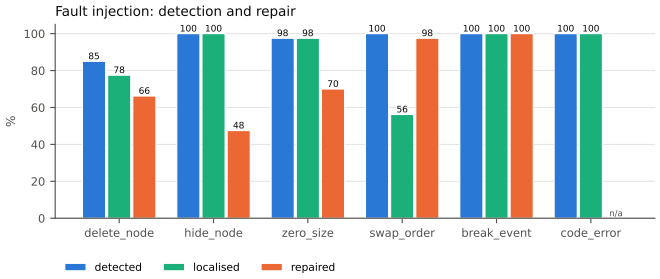


Fig. 5. Fault injection: share of injected faults detected, localised and repaired, by fault type.

are operations on the existing IR, 99.8% of unaffected components are unchanged in type, text, colour, size and font, against 98.2% when the baseline regenerates the program. Latency differs by an order of magnitude: 2.1 s on average for the pipeline (one short interpretation call plus deterministic edit and compile) against 15.4 s for a full regeneration.

E. Validation and Repair

After the development fixes described in Section VI-A, 100.0% of generated UIs pass validation before repair, so on this test set the repair loop is not triggered during normal generation and we cannot measure a pass-rate improvement. We therefore evaluate validation and repair directly by fault injection (Table VI): across 438 injected faults on pipeline and oracle UIs, the validator detects 96.8% and the repair loop restores 73.5%. Detection is strong for hidden, zero-size, reordered and broken-event faults. Repair is reliable for broken event handlers (100%) and reordering (97.5%) but weak for hidden nodes (42.5% on pipeline UIs) and deleted nodes (55%), so the loop is not a general recovery mechanism. Deleting a node is also the fault the validator detects least often on pipeline UIs (75%); we did not investigate why.

F. Generation Time

The proposed pipeline produces a usable UI in 15.5 s on average (median 10.7 s) against 24.0 s for the baseline, including validation and repair (16.4 s). Table VII breaks this down: sketch analysis (10.4 s mean, dominated by the vision call) and interpretation (5.1 s) account for almost all of it, while fusion, code generation and validation together take about 0.5 s. Latencies include network time to the model API and vary with load.

G. Code Quality

Both systems produce code that parses in all cases (Table VIII). Generated code from the proposed compiler is organised in reusable components: each definition is used 1.85 times on average against 1.01 for the baseline, which inlines most elements. The baseline's longest function averages 206 lines against 106 for the proposed code, and the exported project is split into multiple files. ESLint reports almost no messages for either system (0.05 vs. 0.03 per UI). The compiler's longest function grew from 57 to 106 lines between the development run and the final run, after geometry-aware layout was added.

VI. Discussion

A. Development Iteration

The first full evaluation (Run 1) was run on the first complete version of the system and exposed real defects, which we then fixed and re-evaluated on the same test set (Table IX). In Run 1 only 42.5% of generated UIs passed validation; none of the 43 repair actions applied during generation fixed anything; navigation commands succeeded for 3 of 10 cases; the compiler stacked all components in one column, so rendered-DOM detection F1 was 16.5% and layout IoU 20.5%, below the baseline on both. The causes were respectively empty or placeholder-less components that rendered invisibly, a repair step that reset styles instead of fixing the cause, a navigation target the refinement step failed to read from the operation, and a compiler that ignored the sketch geometry already stored in the IR. We fixed these with general changes (placeholder content in fusion, targeted repair actions, geometry-aware layout, global fusion matching) rather than changes specific to individual sketches.

Because the fixes were guided by the same test set, the final numbers are development-set results: the improvement from Run 1 to the final run demonstrates that the

TABLE IV
Generated UI quality: visual similarity, functional correctness, requirement satisfaction

System	Sketches	Renders %	Layout IoU	Edge SSIM	Functional % (n)	Requirements % (n)
Proposed (NeuroGenUI)	40	100.0	0.399	0.522	94.8 (58)	87.5 (407)
Proposed, before repair	40	100.0	0.399	0.522	94.8 (58)	87.5 (407)
Baseline (sketch→AI→UI)	40	100.0	0.283	0.509	86.2 (58)	86.0 (407)
Oracle (GT IR)	40	100.0	0.593	0.551	6.9 (58)	85.3 (407)

TABLE V
Refinement success and preservation (%) per command category

Category	Success: Proposed (NeuroGenUI)	Success: Proposed, Whisper transcript	Success: Baseline (sketch→AI→UI)	Preserved: Proposed (NeuroGenUI)	Preserved: Proposed, Whisper transcript	Preserved: Baseline (sketch→AI→UI)
add	100.0	100.0	80.0	100.0	100.0	96.3
remove	100.0	100.0	100.0	99.4	99.4	98.9
modify	90.0	90.0	100.0	100.0	100.0	100.0
move	80.0	80.0	100.0	100.0	100.0	94.1
resize	90.0	80.0	70.0	99.1	98.2	99.2
style	100.0	100.0	90.0	100.0	100.0	97.1
interaction	100.0	100.0	100.0	100.0	100.0	100.0
navigation	100.0	100.0	80.0	100.0	100.0	100.0
all	95.0	93.8	90.0	99.8	99.7	98.2

TABLE VI
Fault injection: validator detection and repair recovery (%)

IR source	Fault	Cases	Detected %	Localised %	Repaired %
oracle	delete_node	40	95.0	95.0	77.5
oracle	hide_node	40	100.0	100.0	52.5
oracle	zero_size	40	100.0	100.0	65.0
oracle	swap_order	40	100.0	62.5	97.5
oracle	code_error	40	100.0	100.0	None
oracle	all	200	99.0	91.5	73.1
pipeline	delete_node	40	75.0	60.0	55.0
pipeline	hide_node	40	100.0	100.0	42.5
pipeline	zero_size	40	95.0	95.0	75.0
pipeline	swap_order	40	100.0	50.0	97.5
pipeline	break_event	38	100.0	100.0	100.0
pipeline	code_error	40	100.0	100.0	None
pipeline	all	238	95.0	84.0	73.7

TABLE VII
Latency per stage (ms)

System	Stage	n	Mean	Median	p95
pipeline	analyze_sketch	40	10350	5890	10705
pipeline	interpret	40	5114	4987	7215
pipeline	fuse	40	3	3	6
pipeline	codegen	40	1	1	2
pipeline	total_generate	40	15469	10654	18343
pipeline	validate	40	517	443	703
pipeline	repair	40	460	442	667
pipeline	total_with_validation	40	16446	11563	19432
baseline	total_generate	40	23968	16976	65240
analyze_cv	analyze_cv	40	66	55	103
analyze_vision	analyze_vision	40	6445	6237	8711
analyze_hybrid	analyze_hybrid	40	10350	5890	10705
pipeline	modify_total	80	2103	2063	2708
pipeline_voice	modify_total	80	2104	2106	2676
baseline	modify_total	80	15360	13883	23314

evaluation exposes real defects, but it overstates what we should expect on unseen sketches. A held-out set drawn by different people would be required for an unbiased estimate.

B. What the Results Support

The measurements support three claims. (i) Combining OpenCV geometry with a vision model detects more components than either alone, and the structured pipeline recovers more of the sketched components in rendered UIs than a single-call baseline (F1 42.9% vs. 25.6%). (ii) Representing the UI explicitly makes refinement cheap and local: operations on the IR take about 2.1 s, preserve 99.8% of unaffected components and are as successful as full regeneration (95.0% vs. 90.0%, not separable on 80 commands). (iii) Validation in a real browser detects most injected faults (96.8%) and repair restores about three quarters of them. The measurements do not show that the pipeline produces visually closer UIs in an absolute sense (layout IoU 0.40 against a compiler ceiling of 0.59), that it satisfies requirements better (parity), or that the repair loop improves generation pass rates (it was not needed on this set).

C. Limitations

- Synthetic data. Sketches are procedurally rendered in a hand-drawn style and commands are synthesised speech; results on real drawings, varied handwriting and real voices (accents, noise) are expected to be lower.
- Single run, small test set. We report one run on 40 sketches and 80 commands without confidence intervals; model-API variation and heuristic DOM extraction add noise (re-measuring identical cached baseline code moved two baseline numbers by 0.1–0.2 points).
- Tuned on the test set. See Section VI-A.
- Heuristic measurement. Components are recovered from the rendered DOM by tags, roles and visual cues. The procedure is identical for both systems, but it can miss or mis-type components for either.
- One model family. Perception and interpretation use one hosted model and one speech model; the baseline and the pipeline share it, but conclusions may differ for other models.
- Deletion and hidden-node faults are the weakest cases for detection and repair, and sketch detection still misses about 42% of components (recall 58.2% at the analysis stage).

TABLE VIII
Static code quality of the generated React (mean per UI)

Code	UIs	Parses %	LOC	Components	Reuse ratio	Max function lines	ESLint msgs	Files
Proposed (NeuroGenUI) [project]	40	100.0	410.8	13.9	1.85	105.6	0.0	17.9
Proposed (NeuroGenUI) [preview]	40	100.0	379.0	13.9	1.85	105.6	0.1	1.0
Baseline (sketch→AI→UI) [preview]	40	100.0	246.3	1.4	1.01	206.1	0.0	1.0
Oracle (GT IR) [preview]	40	100.0	307.6	11.6	1.60	76.1	1.0	1.0

TABLE IX
Development iteration: the first full evaluation exposed pipeline defects that were fixed (Sec. VI-A); same test set

Metric	Run 1	Final	Baseline
Component detection F1 (analysis vs. baseline DOM)	42.9	53.1	25.6
Component detection F1 (both on rendered DOM)	16.5	42.9	25.6
Visual similarity: layout IoU ×100	20.5	39.9	28.3
Functional correctness	91.4	94.8	86.2
Refinement success rate	81.2	95.0	90.0
Validation pass rate before repair	42.5	100.0	–
Validation pass rate after repair	42.5	100.0	–
Injected faults repaired	60.7	73.5	–

- Scope. The system targets a controlled vocabulary of common components, single-user use, front-end-only behaviour (no backend generation) and English commands.

D. Future Work

Collecting sketches from real users and recorded speech would give an unbiased evaluation. Learned detectors fine-tuned on UI datasets could replace the geometric proposals, and the intent model could be extended with clarification questions for ambiguous commands such as relative resizing. The compiler could target additional frameworks, and the validator could add accessibility and responsiveness checks. A user study of the studio, comparing time to a satisfactory UI against direct prompting, would measure what the latency and preservation advantages mean in practice.

VII. Conclusion

We presented NeuroGenUI, a multimodal framework that turns a hand-drawn sketch and spoken instructions into an editable React interface through an explicit UI Intent Representation, refines it by voice with local operations, and validates and repairs it in a real browser. On 40 synthetic sketches and 80 spoken commands, the staged pipeline detected more of the sketched components in rendered UIs than a direct sketch→AI→UI baseline, satisfied more interactions, and applied refinements about seven times faster with higher preservation, while matching the baseline on requirement satisfaction and refinement success. An initial evaluation also exposed defects that were fixed and re-measured, which we report as a development iteration. The main open problems are detection recall on real hand drawings and robust recovery from deleted or hidden components. The code, dataset and evaluation scripts are provided so that every number in this paper can be reproduced.

References

[1] C. Si, Y. Zhang, Z. Yang, R. Liu, and D. Yang, "Design2code: How far are we from automating front-end engineering?" arXiv:2403.03163, 2024.

[2] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in Proc. International Conference on Machine Learning (ICML), 2023.

[3] J. A. Landay and B. A. Myers, "Interactive sketching for the early stages of user interface design," in Proc. ACM CHI Conference on Human Factors in Computing Systems, 1995.

[4] T. Beltramelli, "pix2code: Generating code from a graphical user interface screenshot," in Proc. ACM SIGCHI Symposium on Engineering Interactive Computing Systems (EICS), 2018.

[5] K. Moran, C. Bernal-Cárdenas, M. Curcio, R. Bonett, and D. Poshyvanyk, "Machine learning-based prototyping of graphical user interfaces for mobile apps," IEEE Transactions on Software Engineering, vol. 46, no. 2, pp. 196–221, 2020.

[6] A. Robinson, "Sketch2code: Generating a website from a paper mockup," arXiv:1905.13750, 2019.

[7] V. Jain, P. Agrawal, S. Banga, R. Kapoor, and S. Gulyani, "Sketch2code: Transformation of sketches to UI in real-time using deep neural network," arXiv:1910.08930, 2019.

[8] B. Deka, Z. Huang, C. Franzen, J. Hibschman, D. Afergan, Y. Li, J. Nichols, and R. Kumar, "Rico: A mobile app dataset for building data-driven design applications," in Proc. ACM Symposium on User Interface Software and Technology (UIST), 2017.

[9] OpenAI, "GPT-4 technical report," arXiv:2303.08774, 2023.

[10] R. A. Bolt, ""put-that-there": Voice and gesture at the graphics interface," in Proc. ACM SIGGRAPH, 1980.

[11] A. Madaan, N. Tandon, P. Gupta et al., "Self-refine: Iterative refinement with self-feedback," in Advances in Neural Information Processing Systems (NeurIPS), 2023.

[12] S. Suzuki and K. Abe, "Topological structural analysis of digitized binary images by border following," Computer Vision, Graphics, and Image Processing, 30(1):32–46, 1985.

[13] G. Bradski, "The OpenCV library," Dr. Dobb's Journal of Software Tools, 2000.

[14] H. W. Kuhn, "The Hungarian method for the assignment problem," Naval Research Logistics Quarterly, vol. 2, no. 1–2, pp. 83–97, 1955.

[15] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," IEEE Transactions on Image Processing, vol. 13, no. 4, pp. 600–612, 2004.

[16] J. Canny, "A computational approach to edge detection," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 8, no. 6, pp. 679–698, 1986.